



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LIBRARY MANAGEMENT SYSTEM

PROJECT REPORT

Submitted by

Kopal Joshi (24BCI0116)

Srijita Kumar (24BCI0113)

Prepared For

DATABASE SYSTEMS (BCSE302P) - LAB PROJECT

Submitted To

Dr. Anand Bihari

School of Computer Science and Engineering

Abstract

Manual handling of book issuing, returns and member records in a college or public library often leads to misplaced registers, delayed fine calculation and difficulty in tracking overdue books. The Library Management System proposed in this project is a relational-database-driven application that automates these day-to-day operations for the library staff and members. It maintains a centralised catalogue of books arranged by category, a record of registered members, a record of the staff who process transactions, and a transaction log that ties books, members and staff together for every issue and return. The system is designed around a normalised MySQL schema so that redundant data is minimised, referential integrity is enforced through foreign keys, and reports such as overdue lists or category-wise book counts can be generated directly through SQL queries. This report documents the scope, ER model, table design with constraints, and the work distribution followed while building the project.

1. Introduction

Libraries, whether in schools, colleges or public institutions, deal with a large and constantly changing collection of books along with a growing base of members who borrow them. Keeping track of this manually using registers is slow, error-prone and makes it hard to answer simple questions such as which books are currently issued, which member has an overdue book, or how many copies of a title are available right now. The Library Management System (LMS) is built to solve exactly this problem by moving all of this information into a structured, relational database. Every book, member, staff member and transaction is stored as a row in a well-defined table, and the relationships between them are enforced by the database itself rather than left to human memory. The project focuses on the database design aspect of the system - identifying the entities involved, the attributes each entity needs, and the relationships that connect them - and then implementing that design in MySQL with appropriate keys, constraints and normal forms.

2. Project Scope

The Library Management System aims at automating the following processes:

- Cataloguing of books along with their category, author, publisher and number of available copies.
- Registration and maintenance of member details such as contact information and date of joining.
- Recording of staff members who are authorised to process book issues and returns.
- Issuing of books to members, along with automatic tracking of issue date and due date.
- Return of books with calculation of fine amount for any overdue days.
- Generation of reports such as currently issued books, overdue books and category-wise book counts using SQL queries.

3. Key Contacts and Stakeholders

Name	Registration Number	Role
Kopal Joshi	24BCI0116	Team Member
Srijita Kumar	24BCI0113	Team Member
Dr. Anand Bihari	-	Faculty Guide

4. Project Resource Requirements

4.1 Software Resource Requirements

- Windows 10 / 11 (or any OS running MySQL)
- MySQL Server 8.0 and MySQL Workbench
- HTML5, CSS3 and JavaScript for the front end
- PHP 8 for server-side scripting
- Apache Web Server (XAMPP)
- draw.io / MySQL Workbench for ER modelling
- Visual Studio Code as the development editor

4.2 Hardware Resource Requirements

- Processor: Intel i3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 2 GB free disk space

5. ER Diagram

The entity-relationship diagram below identifies five entities for the Library Management System: BOOK, CATEGORY, MEMBER, STAFF and the associative entity BOOK_ISSUE, which records every transaction. A book belongs to exactly one category while a category can have many books (1:M). A member can make many transactions over time, and a staff member can process many transactions, so both MEMBER and STAFF relate to BOOK_ISSUE in a 1:M relationship. Similarly, a book can appear in many issue transactions across its lifetime.

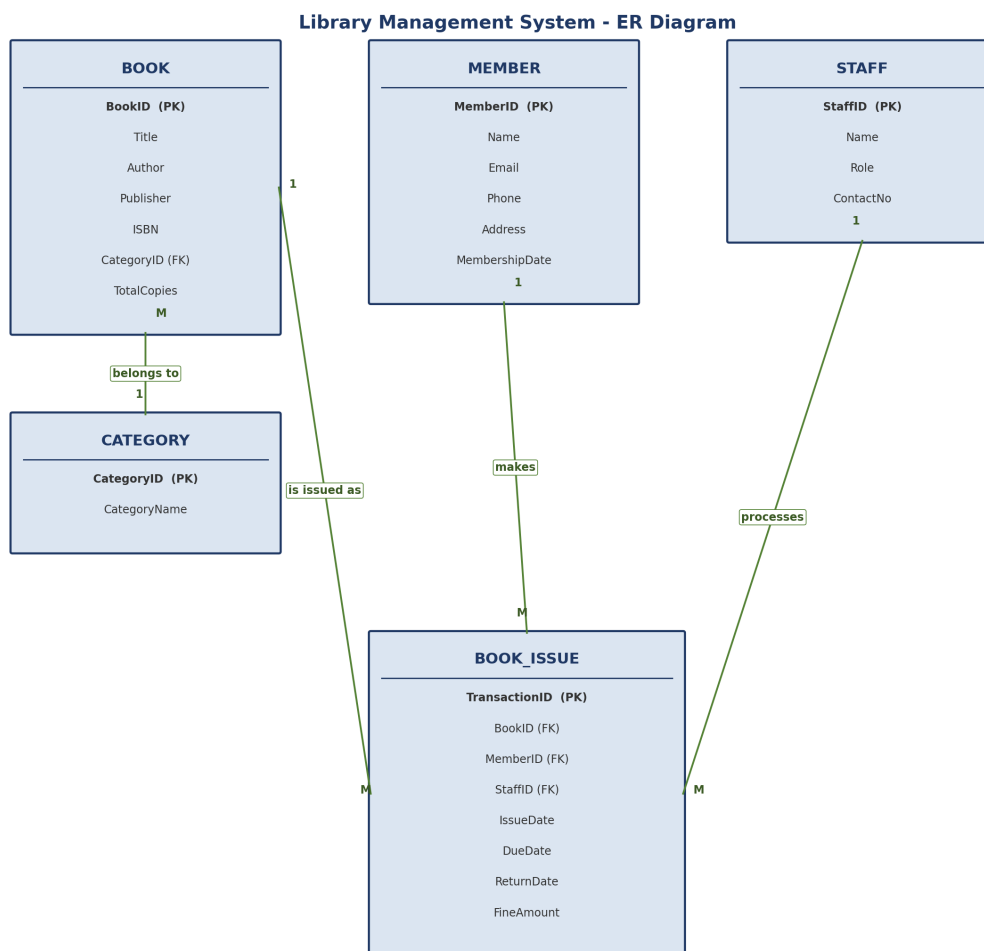


Fig 5.1 - ER Diagram of the Library Management System

6. Tables and Constraints

The schema consists of five tables. Every table is in Third Normal Form (3NF): each column depends only on the primary key of its own table (no partial dependency, since every table other than BOOK_ISSUE uses a single-column primary key), and no non-key column depends on another non-key column (no transitive dependency - for example, CategoryName is stored only in CATEGORY, not repeated inside

BOOK). Foreign keys are used instead of repeating descriptive data, which keeps the design free of redundant, update-prone copies of the same fact.

6.1 CATEGORY

Stores the list of subject categories that books can be classified under, such as Fiction, Reference or Computer Science.

Column	Type / Constraint	Description
CategoryID	INT, PK, AUTO_INCREMENT	Unique identifier for a category.
CategoryName	VARCHAR(50), NOT NULL, UNIQUE	Name of the category.

6.2 BOOK

Stores the catalogue of all books held by the library. CategoryID links each book to exactly one category.

Column	Type / Constraint	Description
BookID	INT, PK, AUTO_INCREMENT	Unique identifier for a book title.
Title	VARCHAR(150), NOT NULL	Title of the book.
Author	VARCHAR(100), NOT NULL	Author's name.
Publisher	VARCHAR(100)	Publishing house.
ISBN	VARCHAR(20), UNIQUE, NOT NULL	International Standard Book Number.
CategoryID	INT, FK -> CATEGORY	Category the book belongs to.
TotalCopies	INT, DEFAULT 1, CHECK >= 0	Number of copies available in the library.

6.3 MEMBER

Stores details of every registered library member who is eligible to borrow books.

Column	Type / Constraint	Description
MemberID	INT, PK, AUTO_INCREMENT	Unique identifier for a member.
Name	VARCHAR(100), NOT NULL	Full name of the member.

Email	VARCHAR(100), UNIQUE, NOT NULL	Contact e-mail, also used for login.
Phone	VARCHAR(15)	Contact phone number.
Address	VARCHAR(200)	Residential address.
MembershipDate	DATE, DEFAULT CURRENT_TIMESTAMP	Date the member joined the library.

6.4 STAFF

Stores details of the library staff who are authorised to issue and receive books.

Column	Type / Constraint	Description
StaffID	INT, PK, AUTO_INCREMENT	Unique identifier for a staff member.
Name	VARCHAR(100), NOT NULL	Full name of the staff member.
Role	VARCHAR(50)	Designation, e.g. Librarian, Assistant.
ContactNo	VARCHAR(15)	Contact phone number.

6.5 BOOK_ISSUE

The associative table that records every book-issue transaction, linking BOOK, MEMBER and STAFF together, and stores the return status and fine for that transaction.

Column	Type / Constraint	Description
TransactionID	INT, PK, AUTO_INCREMENT	Unique identifier for a transaction.
BookID	INT, FK -> BOOK	Book that was issued.
MemberID	INT, FK -> MEMBER	Member who borrowed the book.
StaffID	INT, FK -> STAFF	Staff member who processed the transaction.
IssueDate	DATE, NOT NULL	Date the book was issued.
DueDate	DATE, NOT NULL	Date by which the book must be returned.
ReturnDate	DATE	Actual date of return (NULL until returned).
FineAmount	DECIMAL(6,2), DEFAULT 0.00	Fine charged for late return, if any.

6.6 Table Creation Statements

The screenshots below show the actual CREATE TABLE statements executed in MySQL Workbench, including the primary key, foreign key and check constraints described above.

Fig 6.1 - Creation of CATEGORY and BOOK tables

```

SQL> -- Table: CATEGORY
SQL> CREATE TABLE CATEGORY (
  2     CategoryID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3     CategoryName VARCHAR2(50) NOT NULL UNIQUE
  4 );

-- Table: BOOK
CREATE TABLE BOOK (
  BookID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  Title VARCHAR2(150) NOT NULL,
  Author VARCHAR2(100) NOT NULL,
  Publisher VARCHAR2(100),
  ISBN VARCHAR2(20) UNIQUE NOT NULL,
  CategoryID NUMBER,

Table created.

SQL> SQL> SQL> 2 3 4 5 6 7 8 TotalCopies NUMBER DEFAULT 1 CHECK (TotalCopies >= 0),
9 FOREIGN KEY (CategoryID) REFERENCES CATEGORY(CategoryID)
10 ON DELETE CASCADE
11 );

Table created.

```

Fig 6.2 - Creation of MEMBER and STAFF tables

```

SQL> -- Table: MEMBER
SQL> CREATE TABLE MEMBER (
  2     MemberID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3     Name VARCHAR2(100) NOT NULL,
  4     Email VARCHAR2(100) UNIQUE NOT NULL,
  5     Phone VARCHAR2(15),
  6     Address VARCHAR2(200),
  7     MembershipDate DATE DEFAULT SYSDATE
  8 );

Table created.

SQL> SQL> -- Table: STAFF
SQL> CREATE TABLE STAFF (
  2     StaffID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3     Name VARCHAR2(100) NOT NULL,
  4     Role VARCHAR2(50),
  5     ContactNo VARCHAR2(15)
  6 );

Table created.

```

Fig 6.3 - Creation of BOOK_ISSUE table with foreign key constraints

```

Table created.

SQL> -- Table: BOOK_ISSUE
SQL> CREATE TABLE BOOK_ISSUE (
  2     TransactionID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  3     BookID NUMBER NOT NULL,
  4     MemberID NUMBER NOT NULL,
  5     StaffID NUMBER NOT NULL,
  6     IssueDate DATE NOT NULL,
  7     DueDate DATE NOT NULL,
  8     ReturnDate DATE,
  9     FineAmount NUMBER(6,2) DEFAULT 0.00,
  10    FOREIGN KEY (BookID) REFERENCES BOOK(BookID),
  11    FOREIGN KEY (MemberID) REFERENCES MEMBER(MemberID),
  12    FOREIGN KEY (StaffID) REFERENCES STAFF(StaffID)
  13 );

Table created.

```

7. Work Break Down

Registration Number	Name	Work Assigned
24BCI0116	Kopal Joshi	ER model design, BOOK and CATEGORY table creation, book cataloguing module, report compilation
24BCI0113	Srijita Kumar	MEMBER, STAFF and BOOK_ISSUE table creation, issue/return module with fine calculation, testing

8. Review Evaluation

Component	Marks	Kopal Joshi	Srijita Kumar
ER Diagram	3		
Table creation with description and screenshot	5		
Normal form	2		
TOTAL	10		